



Universidade Federal
do Rio de Janeiro

Escola Politécnica

Arquitetura de Computadores

CPU RV32I

Projeto e implementação de uma CPU RISC-V de 32 bits
com pipeline de 5 estágios em VHDL
com forwarding, detecção de hazards e flush de controle

Alunos: Luiz Cavalini, Eduardo Viana, Henrique Kezen
e Rafael Maurício

Repositório: <https://github.com/LuizCavalini/AC-CPU-RV32I>

Ferramenta: Digital (Neemann) + GHDL

Rio de Janeiro, 8 de julho de 2026

Sumário

1	Introdução	1
2	Arquitetura do Pipeline	1
2.1	Visão geral dos 5 estágios	1
2.2	Diagrama de blocos	2
3	Módulos VHDL	2
3.1	ALU (src/alu/alu.vhd)	2
3.2	Banco de Registradores (src/regfile/regfile.vhd)	2
3.3	Decodificador (src/control/decoder.vhd)	3
3.4	Registradores de Pipeline	3
4	Tratamento de Hazards	3
4.1	Data Hazards — Forwarding	3
4.2	Load-Use Hazard — Stall	4
4.3	Control Hazards — Flush	4
5	Simulação e Verificação	4
5.1	Testbenches individuais	4
5.2	Programa de teste do pipeline	4
5.3	Saída do GHDL	5
5.4	Validação no Digital	5
5.5	Bug encontrado e corrigido: flush incompleto do ID/EX	7
6	Premissas e Decisões de Projeto (Parte 1)	8
7	Extensão Vetorial (Parte 2)	9
7.1	Visão Geral	9
7.2	Codificação das Instruções Vetoriais	9
7.3	Módulos VHDL Novos	10
7.4	Integração no Pipeline	10
7.5	Bugs Encontrados e Corrigidos Durante a Validação	11
7.6	Simulação e Verificação	12
7.7	Validação no Digital	12
7.8	Premissas e Decisões de Projeto da Extensão Vetorial	13
8	Conclusão	14
9	Referências	15

Lista de Figuras

1	Pipeline de 5 estágios com forwarding e detecção de hazards	2
2	Simulação da CPU RV32I no Digital — PC e sinais de depuração . . .	7
3	Simulação da CPU vetorial no Digital — RAM com os 10 resultados confirmados	13

1 Introdução

Este relatório descreve o projeto e a implementação de uma CPU RISC-V de 32 bits (RV32I) com pipeline de 5 estágios em VHDL. O objetivo é construir um processador funcional que execute o subconjunto de instruções inteiras do ISA RISC-V sem modo supervisor, usando pipeline para maximizar o throughput.

A CPU implementa as seguintes instruções:

Tabela 1: Instruções implementadas por categoria

Categoria	Instruções
Aritmética	<code>add</code> , <code>addi</code> , <code>auipc</code> , <code>sub</code>
Lógica	<code>and</code> , <code>andi</code> , <code>or</code> , <code>ori</code> , <code>xor</code> , <code>xori</code>
Deslocamento	<code>sll</code> , <code>slli</code> , <code>srl</code> , <code>srli</code>
Memória	<code>lw</code> , <code>lui</code> , <code>sw</code>
Salto/Desvio	<code>jal</code> , <code>jalr</code> , <code>beq</code> , <code>bne</code>

2 Arquitetura do Pipeline

2.1 Visão geral dos 5 estágios

A CPU é organizada em cinco estágios clássicos separados por registradores de pipeline:

1. **IF (Instruction Fetch):** O PC endereça a memória de instruções e a instrução é carregada no registrador IF/ID.
2. **ID (Instruction Decode):** A instrução é decodificada, os operandos são lidos do banco de registradores e o imediato é gerado por extensão de sinal.
3. **EX (Execute):** A ALU executa a operação. A Forwarding Unit seleciona os operandos corretos. Branches e jumps calculam seus alvos.
4. **MEM (Memory Access):** `lw` lê e `sw` escreve na memória de dados.
5. **WB (Write Back):** O resultado final é escrito no banco de registradores.

2.2 Diagrama de blocos

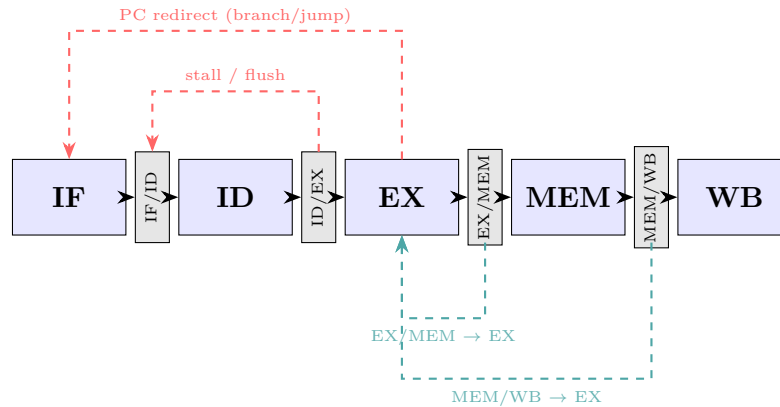


Figura 1: Pipeline de 5 estágios com forwarding e detecção de hazards

3 Módulos VHDL

3.1 ALU (src/alu/alu.vhd)

A ALU de 32 bits recebe dois operandos e um seletor de operação de 4 bits:

Tabela 2: Operações da ALU

alu_ctrl	Operação	Usada por
0000	ADD	add, addi, lw, sw, auipc, jalr
0001	SUB	sub, beq, bne
0010	AND	and, andi
0011	OR	or, ori
0100	XOR	xor, xori
0101	SLL	sll, slli
0110	SRL	srl, srli
0111	LUI	lui (passa B direto)
1000	SLT	comparação com sinal

A flag `zero_o` é ativada quando o resultado é zero, usada por `beq` (verifica `zero=1`) e `bne` (verifica `zero=0`).

3.2 Banco de Registradores (src/regfile/regfile.vhd)

Armazena os 32 registradores de 32 bits. O registrador `x0` é fixo em zero: escritas são ignoradas e leituras sempre retornam `0x00000000`. A escrita é síncrona e a leitura é assíncrona (combinacional), permitindo que WB escreva e ID leia no mesmo ciclo.

3.3 Decodificador (`src/control/decoder.vhd`)

Interpreta opcode, `funct3` e `funct7` para gerar os sinais de controle. Suporta os seis formatos RV32I (R, I, S, B, U, J) com extensão de sinal do imediato para cada formato.

Tabela 3: Sinais de controle gerados pelo decodificador

Sinal	Bits	Descrição
<code>reg_we</code>	1	habilita escrita no banco de registradores
<code>alu_src</code>	1	seleciona <code>rs2</code> (0) ou imediato (1)
<code>alu_ctrl</code>	4	seleciona operação da ALU
<code>mem_we</code>	1	habilita escrita na memória (<code>sw</code>)
<code>mem_re</code>	1	habilita leitura da memória (<code>lw</code>)
<code>wb_sel</code>	2	fonte do writeback (ALU / MEM / PC+4)
<code>branch</code>	1	indica branch (<code>beq/bne</code>)
<code>jump</code>	1	indica salto (<code>jal/jalr</code>)
<code>jalr</code>	1	alvo = <code>rs1+imm</code> em vez de <code>PC+imm</code>
<code>imm</code>	32	imediato com extensão de sinal

3.4 Registradores de Pipeline

- **IF/ID:** propaga PC e instrução. Suporta `flush` (hazard de controle) e `stall` (load-use).
- **ID/EX:** propaga todos os sinais de controle, operandos, imediato, endereços de registradores e PC. Suporta `flush`.
- **EX/MEM:** propaga resultado da ALU, dado de escrita, `rd` e `PC+4`.
- **MEM/WB:** propaga resultado da ALU, dado lido da memória, `rd` e `PC+4` para writeback.

4 Tratamento de Hazards

4.1 Data Hazards — Forwarding

A `forwarding_unit` resolve conflitos RAW encaminhando resultados em trânsito diretamente para as entradas da ALU:

- **EX/MEM → EX:** resultado da ALU do ciclo anterior encaminhado quando `rd_mem = rs1_ex` ou `rd_mem = rs2_ex`.
- **MEM/WB → EX:** dado de writeback encaminhado quando `rd_wb = rs1_ex` ou `rd_wb = rs2_ex`.

- **Prioridade:** EX/MEM tem prioridade sobre MEM/WB.

Codificação dos muxes: "00" = sem forwarding, "01" = MEM/WB, "10" = EX/MEM.

4.2 Load-Use Hazard — Stall

Quando `lw` é seguido imediatamente por instrução que usa o valor carregado, a `hazard_unit` detecta e insere 1 ciclo de stall:

$$\text{stall} = \text{mem_re_ex} \text{ AND } (\text{rd_ex} \neq \text{x0}) \text{ AND } (\text{rd_ex} = \text{rs1_id} \text{ OR } \text{rd_ex} = \text{rs2_id}) \quad (1)$$

A unidade mantém PC e IF/ID inalterados e insere NOP no ID/EX.

4.3 Control Hazards — Flush

Branches e jumps são resolvidos no estágio EX. Quando tomados, o pipeline: (1) redireciona o PC para o alvo, e (2) descarta as duas instruções incorretas já buscadas (flush do IF/ID e ID/EX). Custo fixo: 2 ciclos por branch tomado.

5 Simulação e Verificação

5.1 Testbenches individuais

Tabela 4: Resultado dos testbenches por módulo

Módulo	Casos de teste	Resultado
alu	11	Todos passaram
regfile	5	Todos passaram
decoder	8	Todos passaram
cpu_single_cycle	programa 13 instruções	PC final = 0x44
cpu_pipeline	programa 12 instruções	PC final = 0x5C

5.2 Programa de teste do pipeline

Listing 1: Programa de teste do pipeline (assembly)

```

1 addi x1, x0, 10    -- x1 = 10
2 addi x2, x0, 3     -- x2 = 3
3 add  x3, x1, x2     -- x3 = 13  (forwarding EX/MEM->EX: x1
    disponivel)
```

```

4  sub  x4, x3, x2      -- x4 = 10  (forwarding EX/MEM->EX: x3
    disponivel)
5  addi x5, x0, 13      -- x5 = 13
6  beq  x3, x5, +12     -- x3==x5? sim (13==13), salta para 0x20
7  -- 0x18, 0x1C: SKIPPED (flush por branch tomado)
8  addi x7, x0, 42      -- x7 = 42  (alvo do branch)
9  jal  x8, +8          -- x8 = 0x28, salta para 0x2C
10 -- 0x28: SKIPPED (flush por JAL)
11 addi x10, x0, 55     -- x10 = 55 (alvo do JAL)
12
13 -- extensao: memoria, lui, jalr, logica e deslocamento
14 sw   x3, 0(x0)       -- RAM[0] = 13          (exercita dmem_we)
15 lw   x11, 0(x0)      -- x11 = RAM[0] = 13     (exercita dmem_re)
16 lui  x12, 0x12345    -- x12 = 0x12345000     (exercita lui)
17 addi x16, x0, 0x50   -- x16 = 0x50 (alvo do jalr)
18 jalr x13, x16, 0     -- x13 = 0x44, salta para 0x50
19 -- 0x44, 0x48: SKIPPED (flush por JALR)
20 and  x14, x1, x2     -- x14 = 10 & 3 = 2     (exercita logica)
21 sll  x15, x1, x2     -- x15 = 10 << 3 = 80   (exercita deslocamento)

```

5.3 Saída do GHDL

```

$ ghdl -r --std=08 cpu_pipeline_tb 2>&1
...(report note): CPU pipeline: programa executado com sucesso!
...(report note): PC final = 0x0000005C

$ ghdl -r --std=08 reset_tb 2>&1
...(report note): PC=0x00000014
...(report note): PC apos reset=0x00000004

```

5.4 Validação no Digital

O circuito foi integrado no simulador Digital utilizando o componente *Arquivo externo* com backend GHDL. A estrutura do circuito consiste em:

- **Componente top:** wrapper VHDL que instancia a `cpu_pipeline` e exporta sinais de depuração adicionais.
- **ROM (256 palavras × 32 bits):** carregada com o programa de teste via arquivo `program.hex`.
- **RAM (256 palavras × 32 bits):** memória de dados com portas separadas de leitura e escrita.

- **Distribuidores:** extraem os bits [9:2] dos endereços de 32 bits para endereçar as memórias por palavra.

Os seguintes sinais de depuração são visíveis durante a simulação:

Tabela 5: Sinais de depuração exportados pelo componente top

Sinal	Bits	Descrição
pc_debug_o	32	PC atual (endereço da instrução em IF)
instr_debug_o	32	Instrução sendo decodificada
alu_debug_o	32	Endereço de memória (resultado ALU no MEM)
stall_debug_o	1	Indica stall ativo (PC não avança)
flush_debug_o	1	Indica flush ativo (descarte de instrução após desvio tomado)
imem_addr_o	32	Endereço da memória de instruções
dmem_addr_o	32	Endereço da memória de dados
dmem_wdata_o	32	Dado a escrever na memória (sw)
dmem_we_o	1	Habilitação de escrita na memória
dmem_re_o	1	Habilitação de leitura da memória

A simulação interativa no Digital confirmou os seguintes comportamentos:

Tabela 6: Resultados da simulação interativa no Digital

Teste	Resultado
<i>Fetch e controle de fluxo</i>	
PC avança sequencialmente (0x00 → 0x04 → 0x08 → ...)	Confirmado
Branch tomado (beq): PC busca 0x18 e 0x1C (bolha do pipeline) e redireciona para 0x20 quando o branch resolve no EX	Confirmado
Flush do ID/EX no beq: as instruções buscadas em 0x18/0x1C são descartadas e nunca aparecem em alu_debug	Confirmado
Salto indireto (jalr): PC busca 0x44 e 0x48 (bolha do pipeline) e redireciona para 0x50 quando o jalr resolve no EX	Confirmado
Flush do ID/EX no jalr: idem, para o slot de delay do salto	Confirmado
Decode do jalr: instr_debug = 0x000806E7 em pc_debug = 0x40	Confirmado
Leitura da ROM: instr_debug mostra instruções corretas	Confirmado
<i>Memória de dados</i>	
Escrita (sw): RAM[0x00] = 0x0000000D (13)	Confirmado
Leitura (lw): x11 = 13 (valor lido de volta)	Inferido ¹
<i>Reset</i>	
Reset interativo: PC retorna a 0x00 após pulso de rst_i	Confirmado

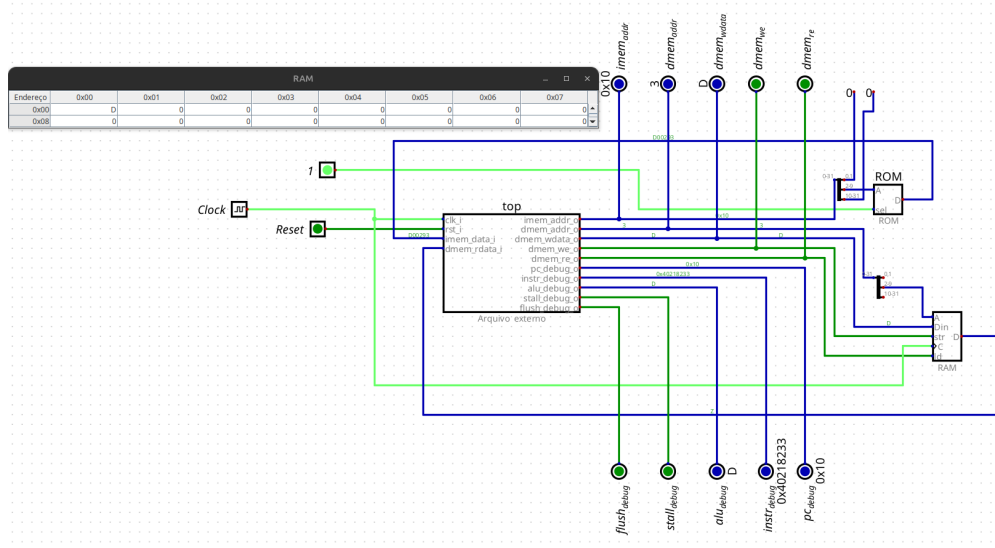


Figura 2: Simulação da CPU RV32I no Digital — PC e sinais de depuração

Observação sobre o reset no Digital: o reset síncrono foi verificado tanto via testbench GHDL (`reset_tb`, Seção 5.3) quanto interativamente no simulador Digital: ao ativar `rst_i` por um ciclo durante a simulação em andamento, o PC retorna corretamente a `0x00` (endereço do vetor de reset), e volta a incrementar normalmente a partir daí (`0x00` → `0x04` → ...) assim que `rst_i` é liberado. O valor `0x00000004` reportado pelo testbench `reset_tb` (Seção 5.3) corresponde a uma amostra do PC um ciclo de clock após o término do reset — ou seja, já incrementado a partir de `0x00` pelo primeiro fetch pós-reset — e não ao valor imediato assumido pelo PC durante o próprio pulso de reset. Os dois resultados são, portanto, consistentes entre si.

5.5 Bug encontrado e corrigido: flush incompleto do ID/EX

Durante a validação interativa no Digital, observou-se que o sinal `flush_debug_o` nunca era ativado durante um branch tomado, apesar do PC ser redirecionado corretamente. Investigação do código-fonte revelou dois problemas:

1. O pino `flush_debug_o` no wrapper `top.vhd` estava conectado a uma heurística derivada de sinais de barramento externos (`dmem_we_o` or `dmem_re_o`), sem relação com o flush real do pipeline.
2. Mais grave: o sinal `flush_idex_s`, que controla o descarte do registrador ID/EX, era acionado apenas pela detecção de load-use hazard (`hazard_unit`), nunca pelo redirecionamento de PC (`pc_redirect_s`) causado por um branch tomado ou jump. Isso significava que, embora o registrador IF/ID fosse corretamente limpo, a instrução do caminho errado que já havia avançado para o ID/EX no mesmo ciclo do redirecionamento escapava do descarte e seria executada indevidamente.

A correção consistiu em: (1) exportar sinais de depuração reais diretamente do `cpu_pipeline` (`pc_if_s`, `instr_id_s`, `alu_result_ex_s`, `stall_pc_s`, `pc_redirect_s`), eliminando as heurísticas do wrapper; e (2) alterar a lógica de flush do ID/EX para `flush_idex_s <= flush_idex_haz_s or pc_redirect_s`, garantindo o descarte tanto em load-use hazard quanto em desvio tomado.

A correção foi validada por um teste dirigido: instruções "veneno" (`addi` com valores identificáveis) foram inseridas nos slots de atraso do `beq` e do `jalr`. Após a correção, `alu_debug_o` nunca exibiu esses valores em nenhum ciclo, confirmando que as instruções de caminho errado são corretamente descartadas. Adicionalmente, os testbenches `cpu_pipeline_tb` e `reset_tb` do GHDL foram reexecutados após a correção (PC final = 0x5C e reset para 0x04, respectivamente), confirmando ausência de regressão.

6 Premissas e Decisões de Projeto (Parte 1)

1. **Branch resolvido no estágio EX.** Reutiliza a ALU (subtração + flag zero) sem hardware adicional, ao custo de 2 ciclos por branch tomado.
2. **Forwarding cobre todos os casos exceto load-use.** EX/MEM→EX e MEM/WB→EX cobrem dependências RAW de 1 e 2 ciclos. Load-use exige 1 stall obrigatório.
3. **Memórias separadas (Harvard).** Conforme o enunciado, memórias de instrução e dados distintas, cada uma entregando uma palavra por ciclo.
4. **Reset síncrono.** Todos os registradores e o PC respondem ao `rst_i` na borda de subida, garantindo inicialização determinística.
5. **Estados internos exportados.** Sinais internos relevantes (flags zero, sinais de forwarding, stall) são expostos como portas para depuração no Digital, conforme exigido pelo enunciado.
6. **AUIPC via mux no operando A.** Quando opcode = 0010111, o operando A da ALU é PC em vez de `rs1`, calculando PC + imm sem hardware adicional.
7. **Espaço de endereçamento limitado a 256 palavras.** ROM e RAM são endereçadas pelos bits [9:2] do barramento de 32 bits (descartando os 2 LSBs, por alinhamento de palavra, e os bits superiores, fixados em zero), resultando em 1KB de espaço endereçável para cada memória. Essa simplificação é adequada ao escopo de um programa de teste da disciplina e não compromete a arquitetura da CPU, que opera sobre o barramento de endereço completo de 32 bits.
8. **CPU inativa durante carga de dados.** Como todos os elementos de estado da CPU (PC e registradores de pipeline) são atualizados exclusivamente na borda de

subida de `clk_i`, e a carga do programa na ROM/RAM ocorre de forma assíncrona pela interface do Digital sem geração de pulsos de clock, o estado interno da CPU permanece inalterado durante o carregamento de dados, atendendo ao requisito do enunciado sem necessidade de lógica adicional.

7 Extensão Vetorial (Parte 2)

7.1 Visão Geral

A segunda etapa do projeto estende a CPU RV32I da Parte 1 para suportar versões vetoriais de oito instruções: `add`, `addi`, `auipc`, `sub`, `sll`, `slli`, `srl`, `srlrli`. A extensão reaproveita o somador vetorial de largura de lane configurável (`cla4/vec_adder`) desenvolvido no projeto AC-AVX-Adder, complementado por um novo deslocador vetorial (`vec_shifter`) com a mesma filosofia de contenção de lane. Todas as 20 instruções escalares da Parte 1 permanecem suportadas sem alteração de comportamento.

7.2 Codificação das Instruções Vetoriais

As oito instruções vetoriais usam o opcode `custom-0` (0001011), reservado pela especificação RISC-V para extensões não padronizadas (RISC-V Unprivileged ISA, tabela de opcodes). O campo `funct3` (`instr[14:12]`) seleciona a operação de forma uniforme entre os oito casos, incluindo o `vauipc`, que no formato U-type padrão não possui esse campo — aqui ele é deliberadamente reaproveitado por uniformidade de decodificação, ao custo de parte do imediato do `vauipc` (ver Tabela 8).

Tabela 7: Codificação `funct3` das instruções vetoriais

<code>funct3</code>	Instrução	Formato	<code>alu_ctrl</code>
000	<code>vadd</code>	R-type-like	0000
001	<code>vaddi</code>	I-type-like	0000
010	<code>vauipc</code>	U-type-like	0000
011	<code>vsub</code>	R-type-like	0001
100	<code>vsll</code>	R-type-like (<code>shamt = rs2[4:0]</code>)	0101
101	<code>vslli</code>	I-type-like (<code>shamt = imm[4:0]</code>)	0101
110	<code>vsrl</code>	R-type-like (<code>shamt = rs2[4:0]</code>)	0110
111	<code>vsrlrli</code>	I-type-like (<code>shamt = imm[4:0]</code>)	0110

Os quatro códigos `alu_ctrl` reaproveitados são exatamente os já existentes na ALU escalar da Parte 1 (Tabela 2) — nenhuma codificação nova foi necessária para o controle da ALU, apenas um novo caminho de decodificação que chega até eles.

Um campo `vecsize` de 2 bits (mesma convenção do `vec_adder` do AC-AVX-Adder: 00=lanes de 4 bits, 01=8 bits, 10=16 bits, 11=32 bits) é extraído de posições distintas conforme o formato:

Tabela 8: Posição do campo `vecsize` e do imediato por formato

Formato	<code>vecsize</code>	Imediato
R-type-like (<code>vadd/vsub/vsll/vsrl</code>)	<code>funct7[1:0]</code>	—
I-type-like aritmética (<code>vaddi</code>)	<code>imm[11:10]</code>	<code>imm[9:0]</code> (10 bits, reduzido de 12)
I-type-like deslocamento (<code>vslli/vsrl</code>)	<code>imm[6:5]</code>	<code>imm[4:0]</code> (shamt, 5 bits, igual ao escalar)
U-type-like (<code>vauipc</code>)	<code>instr[31:30]</code>	<code>instr[29:15]</code> (15 bits, reduzido de 20)

7.3 Módulos VHDL Novos

- **`cla4 (src/vector/cla4.vhd)`**: somador/ subtrator de 4 bits com lógica de carry-lookahead, reaproveitado do projeto AC-AVX-Adder sem alteração de interface.
- **`vec_adder (src/vector/vec_adder.vhd)`**: compõe 8 instâncias de `cla4` em uma cadeia de 32 bits, bloqueando a propagação de carry nas fronteiras de lane conforme `vecsize_i`. Recebeu uma porta de depuração nova, `cout_debug_o`, exportando o carry de cada um dos 8 blocos — resposta direta ao critério de avaliação do enunciado ("um somador pode exportar seu carry para depuração").
- **`vec_shifter (src/vector/vec_shifter.vhd)`**: deslocador de barril (esquerda/direita) implementado por seleção de bit-fonte: para cada bit de saída, calcula-se o bit de entrada correspondente e força-se zero caso a fonte caia fora da lane atual — mesma filosofia de contenção de lane do `vec_adder`, agora aplicada a bits deslocados em vez de carry.
- **`vec_alu (src/vector/vec_alu.vhd)`**: módulo topo que multiplexa entre `vec_adder` (para `vadd/vaddi/vauipc/vsub`) e `vec_shifter` (para `vsll/vslli/vsrl/vsrl`) com base no mesmo código `alu_ctrl` da ALU escalar.

7.4 Integração no Pipeline

O `decoder` foi estendido com dois novos sinais de saída, `is_vector_o` e `vecsize_o`, através de um novo ramo no `case opcode` para `custom-0`. O registrador `id_ex_reg` passou a propagar ambos os sinais de ID para EX, com o mesmo comportamento de `flush` (zeramento) dos demais sinais de controle — garantindo que uma instrução descartada por hazard de controle nunca dispare o `vec_alu` indevidamente.

No estágio EX, uma instância de `vec_alu` opera em paralelo à ALU escalar, consumindo os mesmos operandos já multiplexados (`alu_a_muxed_s/alu_b_muxed_s`),

que já contemplam forwarding e o caso especial do `auipc` (PC como operando A). Um novo sinal, `alu_result_muxed_s`, seleciona entre o resultado escalar e o vetorial conforme `is_vector_ex_s`, e é esse sinal combinado — não mais `alu_result_ex_s` diretamente — que alimenta o registrador EX/MEM e a saída de depuração `alu_debug_o`. Como o *forwarding* já opera a partir da saída do registrador EX/MEM (não da ALU diretamente), a `forwarding_unit` e a `hazard_unit` não precisaram de nenhuma alteração — o encaminhamento de resultados vetoriais funciona "de graça".

Adicionalmente, o sinal `auipc_id_s` (que seleciona PC como operando A da ALU) precisou ser estendido para também reconhecer o `vauipc`, já que originalmente comparava apenas contra o opcode escalar do `auipc`.

7.5 Bugs Encontrados e Corrigidos Durante a Validação

A construção do testbench de integração da Parte 2 revelou dois bugs pré-existentes, ambos independentes da extensão vetorial em si.

Hazard RAW de distância-3 no banco de registradores. O relatório da Parte 1 já afirmava que "a leitura é assíncrona, permitindo que WB escreva e ID leia no mesmo ciclo", mas a implementação não cumpria essa promessa: a escrita em `regs()` só se tornava visível à leitura combinacional um ciclo de clock depois de `we_i/ wd_i` estabilizarem. Isso deslocava a "transparência" de distância-3 para distância-4, deixando dependências RAW de distância-3 sem cobertura — longe demais para os caminhos de 1 e 2 ciclos da `forwarding_unit`, um ciclo cedo demais para a auto-resolução do banco de registradores. O bug nunca havia sido detectado porque o testbench original da Parte 1 só continha dependências de distância 1 e 2. Confirmado com um repro escalar isolado (independente de qualquer instrução vetorial) e corrigido com um bypass clássico de escrita-leitura no mesmo ciclo em `regfile.vhd`.

Encadeamento de carry em subtração vetorial multi-bloco. O `cla4.vhd` injetava o "+1" do complemento de dois incondicionalmente em todo bloco sempre que `sub_i='1'` (`c(0) <= cin_i or sub_i`), em vez de fazê-lo apenas no primeiro bloco de uma cadeia. Isso corrompia qualquer subtração vetorial com lane maior que 4 bits sempre que o borrow real entre blocos deveria ser zero. O bug foi isolado por rastreamento sinal-a-sinal ciclo a ciclo (todos os sinais de controle e operandos do `vec_alu` confirmados corretos no ciclo da falha), o que restringiu a causa à lógica interna do somador. A correção moveu a injeção do "+1" do `cla4` (que agora apenas honra o `cin_i` recebido, como um somador em cascata convencional) para o `vec_adder`, que injeta o "+1" apenas nas fronteiras de início de lane, propagando o carry real sem interferência dentro de uma lane aberta.

Ambos os bugs foram corrigidos com regressão completa (todos os testbenches

pré-existentes revalidados) e são detalhados nos commits 2fcfb7a e d58572b do repositório.

7.6 Simulação e Verificação

Tabela 9: Resultado dos testbenches da extensão vetorial

Módulo	Casos de teste	Resultado
cla4	5	Todos passaram
vec_adder	8	Todos passaram
vec_shifter	7	Todos passaram
vec_alu	4	Todos passaram
decoder (casos vetoriais)	8	Todos passaram
cpu_pipeline_vector_tb	programa de 29 instruções	10/10 valores corretos

O `cpu_pipeline_vector_tb` executa um programa misto (construção de operandos via instruções escalares já validadas na Parte 1, seguidas das 8 instruções vetoriais em múltiplas configurações de `vecsize`) e verifica os resultados via escrita em memória (`sw`), incluindo um caso de *overflow* sem vazamento entre lanes de 4 bits, um caso de dependência imediata entre duas instruções vetoriais (`vadd` seguido de `vsub`, testando forwarding de resultado vetorial) e um contraste entre deslocamento contido em lane e deslocamento de largura total.

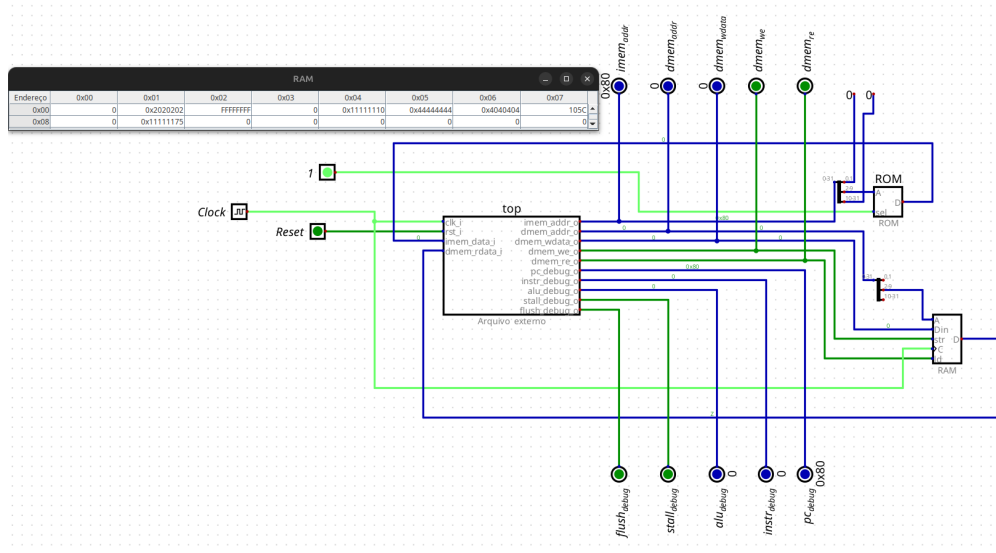
7.7 Validação no Digital

O componente `top` do Digital já expunha os cinco sinais de depuração da Parte 1; um sexto, `vec_cout_debug_o`, foi adicionado para o carry vetorial. Como o mecanismo de build do Digital (compilação sob demanda contra uma biblioteca de trabalho do GHDL, não mais o arquivo único `cpu_pipeline_full.vhd` usado inicialmente na Parte 1) não possuía um processo documentado e reproduzível, foi criado `digital/build.sh`, testado com sucesso a partir de uma biblioteca de trabalho vazia.

O mesmo programa de 29 instruções do testbench de GHDL foi carregado na ROM (`digital/program_vector.hex`) e executado interativamente no Digital. Os 10 resultados na RAM conferem exatamente com os obtidos via GHDL:

Tabela 10: Resultados do programa vetorial confirmados na RAM (Digital)

Instrução	Endereço RAM	Valor
vadd (4b, overflow)	0x00	0x00000000
vsub (8b)	0x04	0x02020202
vsub (32b, forwarding)	0x08	0xFFFFFFFF
vslli (4b, contido)	0x0C	0x00000000
vslli (32b)	0x10	0x11111110
vsll (8b, reg-reg)	0x14	0x44444444
vsrl (8b, reg-reg)	0x18	0x04040404
vauipc	0x1C	0x0000105C
vsrli (4b, contido)	0x20	0x00000000
vaddi (8b)	0x24	0x11111175

**Figura 3:** Simulação da CPU vetorial no Digital — RAM com os 10 resultados confirmados

Os endereços 0x00 e 0x08 correspondem exatamente às instruções que expuseram os dois bugs corrigidos na Seção 6.5, servindo como confirmação visual das correções fora do ambiente de simulação de linha de comando.

7.8 Premissas e Decisões de Projeto da Extensão Vetorial

1. **Opcode custom-0.** Escolhido por ser explicitamente reservado pela especificação RISC-V para extensões não padronizadas, sem conflito com nenhum opcode escalar da Parte 1.
2. **Reaproveitamento da convenção vecsize do AC-AVX-Adder.** Os mesmos quatro valores (4/8/16/32 bits) usados no somador vetorial da atividade AVX foram adotados aqui, mantendo consistência entre os dois projetos.

3. **funct3 uniforme mesmo para `vauipc`.** O formato U-type padrão não possui campo `funct3`; reaproveitar essa posição de bits para o `vauipc` mantém a lógica de decodificação uniforme entre os oito casos, ao custo de reduzir o imediato do `vauipc` de 20 para 15 bits.
4. **Imediatos reduzidos.** `vaddi` tem imediato de 10 bits (vs. 12 no `addi` escalar) e `vauipc` de 15 bits (vs. 20 no `auipc` escalar), ambos para acomodar o campo `vecsize`. Adequado ao escopo de um programa de teste da disciplina.
5. **Semântica do `vauipc`.** Como PC é um valor escalar único, "vetorizar" `auipc` não tem significado físico pleno para lanes menores que 32 bits. A implementação roteia PC e o imediato pelo mesmo `vec_adder` usado por `vadd/vsub`, com `vecsize` controlando apenas onde o carry é quebrado — funcionalmente consistente com a categoria aritmética, mesmo que o resultado com lanes <32 bits não tenha uso prático como endereço.
6. **Reaproveitamento dos códigos `alu_ctrl`.** Nenhuma codificação de controle de ALU nova foi criada; o `vec_alu` usa exatamente os mesmos quatro códigos já existentes na ALU escalar da Parte 1, simplificando a integração no decoder.

8 Conclusão

O projeto resultou em uma CPU RV32I funcional com pipeline de 5 estágios, implementando as instruções obrigatórias. A validação em GHDL confirmou o funcionamento correto do forwarding, detecção de load-use e flush de control hazards. O PC final de 0x5C no testbench do pipeline confirma que branches tomados e JAL foram executados corretamente com descarte das instruções incorretas.

A extensão vetorial (Parte 2) adicionou oito instruções customizadas via opcode `custom-0`, reaproveitando o somador vetorial do projeto AC-AVX-Adder e um novo deslocador de barril com a mesma filosofia de contenção de lane. A integração revelou e corrigiu dois bugs pré-existentes — um hazard RAW de distância-3 no banco de registradores e um erro de encadeamento de carry em subtrações vetoriais multi-bloco — ambos validados com regressão completa. As 8 instruções vetoriais foram confirmadas corretas tanto via GHDL (10/10 casos) quanto interativamente no Digital (10/10 valores na RAM), incluindo os dois casos que expuseram os bugs corrigidos.

O código-fonte está disponível em <https://github.com/LuizCavalini/AC-CPU-RV32I>.

9 Referências

1. Patterson, D. A. & Hennessy, J. L. *Computer Organization and Design: RISC-V Edition*, 2^a ed., Elsevier, 2021. Capítulos 4 e 5.
2. RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, versão 20191213. <https://riscv.org/technical/specifications/>
3. Neemann, H. *Digital — Logic Designer and Circuit Simulator*. <https://github.com/hneemann/Digital>
4. Pedroni, V. A. *Circuit Design and Simulation with VHDL*, 2^a ed., MIT Press, 2010.